

Reviewer Pack — Minimal Local Index of Topological Entanglement Rank and Com...

Entry 2e208f9659c3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 13:41:05 UTC

Minimal Local Index of Topological Entanglement Rank and Communication Complexity

Entry ID: 2e208f9659c3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 13:41:05 UTC

1. Conjecture

Field A (mathematical branch): Topological Quantum Field Theory **Field B** (complexity object): Communication Complexity

Statement:

The minimal local index of topological entanglement rank ($mter(X)$) for a bipartite quantum state X is upper-bounded by the communication complexity ($cc(X)$) of a protocol that distributes X , such that $mter(X) = O(cc(X))$.

Rationale (proposer's reasoning):

Topological quantum field theories provide a framework for understanding quantum entanglement in terms of topological properties. If the minimal local index of topological entanglement rank can be shown to relate to communication complexity, it may reveal a deep connection between quantum information and classical computation.

Taxonomy category: topological_quantum_field_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 57cd6c5a57c00690

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between $mter(X)$ and $cc(X)$ for all generated states X is ≥ 0.9 , with p-value ≤ 0.05 over at least 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - topological quantum field theory AND minimal local index OF topological entanglement rank - communication complexity AND upper bound ON minimal local index OF topological entanglement rank - $\text{mter}(X) = 0(\text{cc}(X))$ AND quantum state distribution

Top relevant hits considered: - [http://arxiv.org/abs/1610.07628v2] Higher Categories and Topological Quantum Field Theories - [http://arxiv.org/abs/1708.04619v5] Quantum Field Theory of X-Cube Fracton Topological Order and Robust Degeneracy from Geometry - [http://arxiv.org/abs/1811.02143v1] Symmetry defects and their application to topological quantum computing - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1109.2952v4] Upper bound on distance in the pants complex - [http://arxiv.org/abs/1409.4035v2] The Corruption Bound, Log Rank, and Communication Complexity - [http://arxiv.org/abs/2502.20821v2] Improved measurement of absolute branching fraction of the inclusive decay $\Lambda_c^+ \rightarrow K_S^0 X$ - [http://arxiv.org/abs/2403.16811v2] Cross section measurement of $e^+e^- \rightarrow \eta\psi(2S)$ and search for $e^+e^- \rightarrow \eta\tilde{X}$ (3872)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bipartite_state(n):
        # Generate a random bipartite quantum state with entanglement rank up to n
        state = [[random.random() for _ in range(n)] for _ in range(n)]
        return state

    def mter(state):
        # Compute the minimal local index of topological entanglement rank (mter(X))
        # This is a placeholder function. Replace with actual algorithm.
        return sum(sum(row) for row in state)

    def cc(state):
        # Determine the communication complexity (cc(X)) of a protocol that distributes X
        # This is a placeholder function. Replace with actual algorithm.
        n = len(state)
        return n * (n - 1) / 2
```

```

total_mter = []
total_cc = []

for _ in range(30):
    state = generate_bipartite_state(random.randint(5, 40))
    mter_value = mter(state)
    cc_value = cc(state)
    total_mter.append(mter_value)
    total_cc.append(cc_value)

instances_tested = len(total_mter)
n_max = max(len(state) for state in [generate_bipartite_state(n) for n in range(5, 41)])
correlation_coefficient = (instances_tested * sum(mter * cc for mter, cc in zip(total_mter, total_cc)) -
                           instances_tested * sum(total_mter) * sum(total_cc)) / \
    ((instances_tested * sum(mter**2 for mter in total_mter) - sum(total_mter)**2) *
     (instances_tested * sum(cc**2 for cc in total_cc) - sum(total_cc)**2))

conjecture_holds = correlation_coefficient >= 0.9
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))**0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
erexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -33.37870196557317, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -54.017637297379345, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -38.189608222484694, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -82.6254277325861, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -43.60359235667952, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -71.17447644623373, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -30.705656127097853, 'instances_t
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -37.92461164221331, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -68.94485382131658, 'instances_te
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': -33.22525545961583, 'instances_te
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder functions for both mter(state) and cc(state), which are critical components of the conjecture. Without actual implementations, it is impossible to confirm whether the conjecture holds or not. The correlation coefficient metric used as a substitute does not directly measure the relationship between mter(X) and cc(X).

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code uses placeholder functions for both mter(state) and cc(state), which are critical components of the conjecture. The correlation coeffici | next: Develop proper implementations for mter(state) and cc(state), then retest the conjecture with accurate metrics.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14925
2	propose	ollama_remote	glm4:latest	0	0	11706
3	propose	ollama_remote	glm4:latest	0	0	13502
4	preregistration	ollama_remote	glm4:latest	0	0	9159
5	novelty	ollama_remote	glm4:latest	0	0	8345
6	novelty	ollama_remote	glm4:latest	0	0	16723
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26491
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16399
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17154
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25365
11	critic	ollama_remote	glm4:latest	0	0	14771
12	judge	ollama_remote	glm4:latest	0	0	13548

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 188087 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/2e208f9659c3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2e208f9659c3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2e208f9659c3.tar.gz` (if generated)